

If we have a sequence of m `insert`, `merge`, or `delete_min` operations, each of them exchanging `left` and `right` in k_i nodes, of which $k_i^{\mathcal{L}}$ are left-heavy and $k_i^{\mathcal{R}}$ are right-heavy, then the total time is $O(m + \sum_{i=1}^m k_i) = O(m + \sum_{i=1}^m k_i^{\mathcal{L}} + \sum_{i=1}^m k_i^{\mathcal{R}}) = O(m + m \log n) + O(\sum_{i=1}^m k_i^{\mathcal{R}})$. To bound the last sum, we notice that the potential is initially at most n , in the end it is at least 0, we subtract $\sum_{i=1}^m k_i^{\mathcal{R}}$ and add $\sum_{i=1}^m k_i^{\mathcal{L}} \leq m \log n$; thus, $\sum_{i=1}^m k_i^{\mathcal{R}} \leq n + m \log n$. The total time of m operations on n elements is therefore $O(n + m \log n)$. Together this shows the following:

Theorem. The skew heap structure supports the operations `find_min` in $O(1)$ time and `insert`, `merge`, and `delete_min` in amortized $O(\log n)$ time on a heap with n elements.

A more complicated variant, the bottom-up skew heaps, was also described in Sleator and Tarjan (1986); they achieve `insert` and `merge` in $O(1)$ amortized time. Because they contain additional pointers that need to be updated in a `delete_min` operation, for bottom-up skew heaps we cannot use the reduction of `delete_min` to `merge`; the `delete_min` operation still has $O(\log n)$ amortized complexity. None of these complexities are worst case; at worst they could be $\Omega(n)$. The structure was somewhat further studied in Jones (1987), Kaldewaij and Schoenmakers (1991), and Schoenmakers (1997).

5.6 Binomial Heaps

Binomial heaps are another classical, although somewhat complicated, method to achieve all heap operations including `merge` in $O(\log n)$ time. In contrast to the previous structure, the `find_min` operation also needs $\Theta(\log n)$ time. Binomial heaps were invented by Vuillemin (1978) and are mainly interesting for another type of additional operation, the change of key values, which will require a separate discussion in a later section.

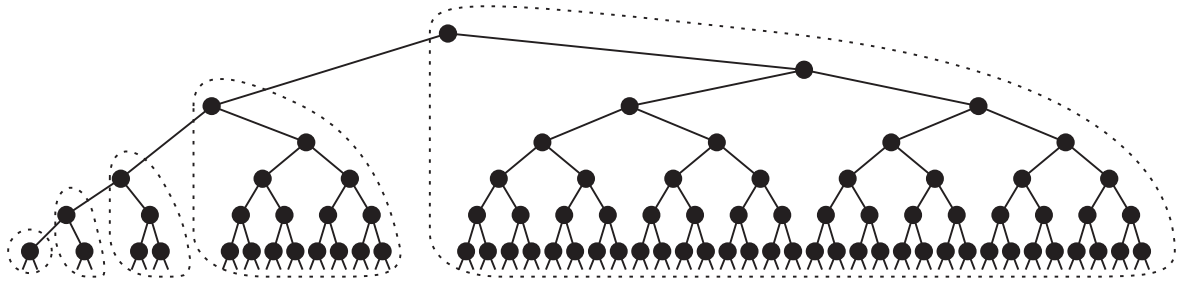
Binomial heaps can again be written as binary trees with keys and objects in each node, but they are not heap-ordered trees, but only half-ordered trees:

1. If node w is in the right subtree of node v , then $v \rightarrow \text{key} < w \rightarrow \text{key}$.

This is a weaker condition than heap order: keys get larger to the right, but on a left path keys might appear in any order. The minimum key itself might occur anywhere along the path from the root to the left. This weaker order condition is coupled with a stronger balance condition.

2. If v is a node on the path from the root to the left, then $v \rightarrow \text{right}$ is root of a complete binary tree. The height of these trees is strictly decreasing along the path from the root to the left.

Thus the binomial heap consists of blocks of the following structure, which are put together on the left path: a node n_i on the path, whose `right` pointer points to a complete binary tree of height h_i , where this h_i is decreasing along the path. The complete binary tree of height h contains $2^{h+1} - 1$ nodes, so together with the node on the leftmost path the block has 2^{h+1} nodes. We also allow the empty tree as complete binary tree of height -1 , so there might also be a block of $2^0 = 1$ node, just consisting of the node on the path. If the block sizes along the path are $2^{h_1} > 2^{h_2} > \dots > 2^{h_k}$, then $2^{h_1} + 2^{h_2} + \dots + 2^{h_k} = n$, so the block decomposition corresponds to the binary expansion of the total size n of the heap.

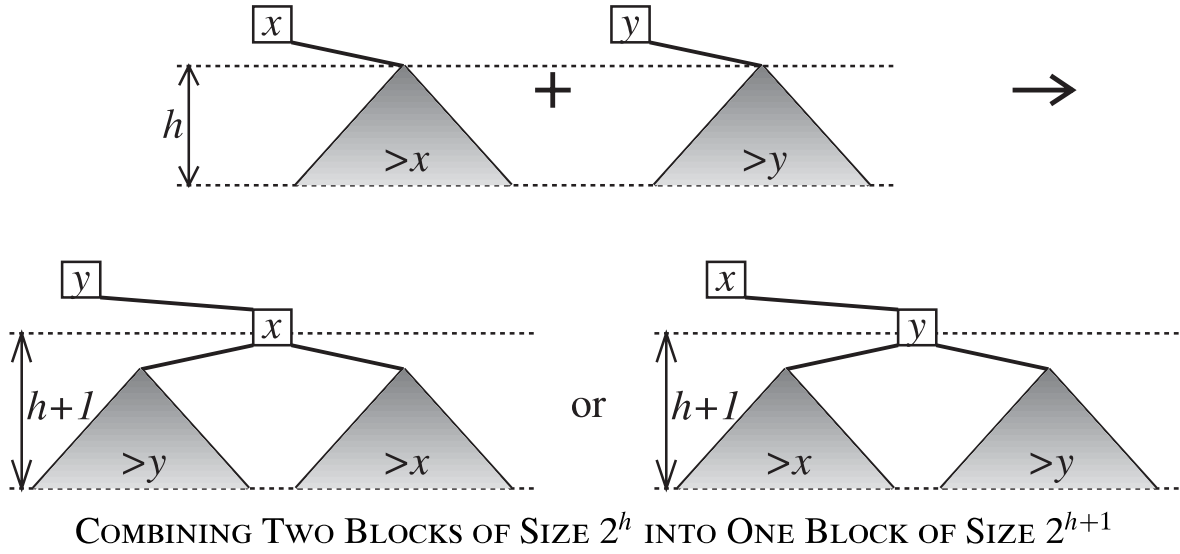


BINOMIAL HEAP STRUCTURE, WITH BLOCKS OF SIZE 2^0 , 2^1 , 2^2 , 2^4 , AND 2^6

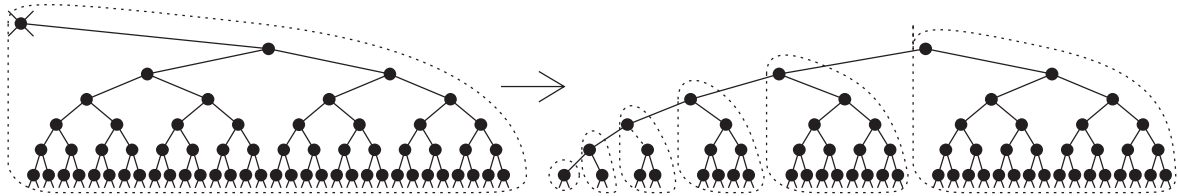
The central property of these blocks is that one can combine in time $O(1)$ two blocks of the same size 2^h into one block of size 2^{h+1} : if n and m are the top nodes of two blocks, for which both $n \rightarrow \text{right}$ and $m \rightarrow \text{right}$ are complete binary trees of height h and $n \rightarrow \text{key} < m \rightarrow \text{key}$, then we can make n the new top node, whose `right` field points to m , and m becomes root of a complete binary tree of height $h + 1$, with the tree previously below $n \rightarrow \text{right}$ now below $m \rightarrow \text{left}$. This is the point where the weaker order condition 1 is needed; if we required heap order, we could not just join these trees together because the heap-order relation between m and the new $m \rightarrow \text{left}$ could be violated, but condition 1 does not require any order along the left paths.

With this “adding” of two individual blocks in $O(1)$ time, we can merge two binomial heaps by performing an addition with carry of the two left paths.

The other operations, `insert` and `delete_min`, can be reduced to merge. An `insert` is just a merge with a single-node heap. For a `delete_min`, we have to find the minimum node from the leftmost path of the root, unlink that block, and delete its top node. Then the remaining block is a complete binary tree, which itself is a binomial heap, so it can be merged



back with the original heap from which it was removed to get the heap resulting from the deletion.



ONE BLOCK MINUS ITS ROOT IS AGAIN A BINOMIAL HEAP

We get an $O(\log n)$ bound for all operations, `insert`, `merge`, and `delete_min`, from observing that the binary addition with carry of two $\log n$ -bit numbers requires $O(\log n)$ bit additions. This bound can be improved, in the amortized sense, for long sequences of `insert` operations, which correspond to repeated adding of 1 or counting in binary numbers. The total number of bits flipped while counting in binary from n to $n + i$ is $O(i + \log n)$, so the complexity of i consecutive insertions in a heap of initial size n is $O(i + \log n)$. Consider now a sequence of a `insert` and b `merge` or `delete_min` operations on a set of heaps with initial total size n , which at the end consists of k heaps. For each heap, each `delete_min` or `merge` operation, of complexity $O(\log n)$, is preceded by some sequence of a_j `insert` operations, of complexity $O(a_j + \log n)$. There are also some a_{final} `insert` operations that are not followed by any `delete_min` or `merge` involving that heap; these take at most $O(a_{\text{final}} + k \log n)$ time. Because $a = a_{\text{final}} + \sum_j a_j$, this gives a total complexity of $O(a + b \log n + k \log n)$. Because $k \leq n$, this shows that the amortized complexity of the `insert` operation is $O(1)$.

Next we give the code for the `merge` of two binomial heaps, as well as some elementary operations. Because the size of the blocks is decreasing along the paths, but we have to add starting from the blocks of smallest size, we put in a first phase just all blocks on a stack to invert the order. An alternative solution

would be to put the blocks in increasing size on the leftmost path; in that case the blocks themselves would not be complete binary trees but very skewed objects of twice the height. The total work is the same, and it is preferable to use standard binary trees. The required height of the stack is only $2 \log n$.

```
typedef struct hp_n_t {
    int          height;
    key_t        key;
    object_t     *object;
    struct hp_n_t *left;
    struct hp_n_t *right;
} heap_node_t;

heap_node_t *create_heap(void)
{ heap_node_t *tmp_node;
  tmp_node = get_node();
  tmp_node->height = -1;
  tmp_node->left = tmp_node->right = NULL;
  return( tmp_node );
}

int heap_empty(heap_node_t *hp)
{ return( hp->height == -1 );
}

key_t find_min_key(heap_node_t *hp)
{ heap_node_t *tmp;
  key_t      tmp_key;
  tmp       = hp;
  tmp_key = hp->key;
  while( tmp->left != NULL )
  { tmp = tmp->left;
    if( tmp->key < tmp_key )
      tmp_key = tmp->key;
  }
  return( tmp_key );
}

heap_node_t *merge( heap_node_t *hp1,
                    heap_node_t *hp2)
```

```

{  heap_node_t *tmp1, *tmp2, *current, *next;
   if( hp1->height == -1 ) /* heap 1 empty */
   {  return_node(  hp1 );
       return( hp2 );
   }
   if( hp2->height == -1 ) /* heap 2 empty */
   {  return_node(  hp2 );
       return( hp1 );
   }
   /* put all the blocks on the stack */
   create_stack();
   tmp1 = hp1; tmp2 = hp2;
   while( tmp1 != NULL && tmp2 != NULL )
   {  if( tmp1->height > tmp2->height )
       {  push( tmp1 );
           tmp1 = tmp1->left;
       }
       else
       {  push( tmp2 );
           tmp2 = tmp2->left;
       }
   }
   /* one list is empty, push the rest
      of the other */
   while( tmp1 != NULL )
   {  push( tmp1 );
       tmp1 = tmp1->left;
   }
   while( tmp2 != NULL )
   {  push( tmp2 );
       tmp2 = tmp2->left;
   }
   /* now all the blocks are on the stack */
   /* put them together, performing addition */
   current = pop();
   while( !stack_empty )
   {  next = pop();
       if( next->height > current->height )
       {  next->left = current;

```

```

        /* add in front of left list */
        current = next;
    }
    else if( next->height ==
            current->height ) /* add blocks */
    {
        if( next->key < current->key )
        {
            next->left = current->left;
            current->left = next->right;
            next->right = current;
            next->height += 1;
            current = next;
        }
        else
        {
            next->left = current->right;
            current->right = next;
            current->height +=1;
        }
    }
    else /* next->height < current->height */
    {
        next->left = current->left;
        /* exchange current, next*/
        current->left = next;
        /* insert next just below current */
    }
}
return( current );
}

```

The `delete_min` code needs some care, because we have to preserve the address of the root node, and the result of the merge could be a different node. The same problem exists for the `insert`. Here we just copy the root to a different node and then copy the result back. This could be avoided if we used a placeholder node above the root. The placeholder could also point to the current minimum node on the leftmost path, making `find_min` an $O(1)$ operation by moving the minimum update time to the `insert` and `delete_min` operations. For the greater regularity of the structure, we decided to avoid a placeholder node. Next we give the code for the `delete_min` and `insert` operations.

```

object_t *delete_min(heap_node_t *hp)
{
    object_t *del_obj;
    heap_node_t *tmp1, *tmp2, *min1, *min2;
    key_t      tmp_key;
    if( hp->height == 0 )
        /* delete last object, heap now empty */
    {
        hp->height = -1;
        return( hp->object );
    } /* can assume now that heap will
        not become empty */
    tmp1 = tmp2 = hp;
    tmp_key = hp->key;
    min1 = min2 = hp;
    while( tmp1->left != NULL )
    {
        tmp2 = tmp1;
        /* tmp2 node above tmp1 on left path */
        tmp1 = tmp1->left;
        if( tmp1->key < tmp_key )
        {
            tmp_key = tmp1->key;
            /* min1 is minimum node */
            min1 = tmp1; min2 = tmp2;
            /* min2 node above min1 */
        }
    }
    del_obj = min1->object;
    if( min1 != min2 ) /* min1 not root,
so node above exists */
    {
        min2->left = min1->left;
        /* unlink min1 */
        if( min1->height > 0 )
            /* min1 has right subtree */
        {
            tmp1 = min1->right;
            /* save its right tree */
            min1->key = hp->key;
            /* copy root into min1 */
            min1->object = hp->object;
            min1->height = hp->height;
            min1->left = hp->left;
            min1->right = hp->right;
            tmp2 = merge( min1, tmp1 );
        }
    }
}

```

```

        /* and merge */
    }
    else /* min1 is leaf on left path */
    {   return_node( min1 );
        return( del_obj );
    }
} /* min1 is root node, has left
   and right subtrees */
else if ( min1->left != NULL )
    tmp2 = merge( min1->left, min1->right );
else /* min1 is root node, has only
      right subtree */
    tmp2 = min1->right;
/* merge completed, now copy new root back */
hp->key      = tmp2->key;
hp->object   = tmp2->object;
hp->height   = tmp2->height;
hp->left     = tmp2->left;
hp->right    = tmp2->right;
return_node( tmp2 );
return( del_obj );
}

```

```

int insert( key_t new_key, object_t *new_obj,
            heap_node_t *hp)
{   heap_node_t *new_node, *tmp, *tmp2;
    new_node = get_node();
    /* create one-element heap */
    new_node->height = 0;
    new_node->key     = new_key;
    new_node->object  = new_obj;
    new_node->left    = new_node->right = NULL;
    tmp = get_node();
    /* copy root into tmp_node */
    tmp->left     = hp->left;
    tmp->right    = hp->right;
    tmp->key      = hp->key;
    tmp->object   = hp->object;

```



```

tmp->height    = hp->height;
tmp2 = merge( new_node, tmp );
/* merge the heaps */
hp->left = tmp2->left;
/* merge completed, copy root back */
hp->right = tmp2->right;
hp->key = tmp2->key;
hp->object = tmp2->object;
hp->height = tmp2->height;
return_node( tmp2 );
return( 0 );
}

```

To summarize the performance of this structure, we have the following:

Theorem. The binomial heap structure supports the operations `insert`, `merge`, `find_min`, and `delete_min` in $O(\log n)$ time.

The amortized complexity of the `insert` operation is $O(1)$; any sequence of a `insert` and b `delete_min` or `merge` operations on a set of heaps of initial total size n , with k heaps remaining at the end, takes $O(a + b \log n + k \log n)$ time.

The key idea of the binomial heap structure is this decomposition of the heap into these blocks of canonical size (2^i for some i) that are guaranteed to have small height and that can be combined to the next larger size in constant time. With this block structure, we can then merge two heaps by performing binary addition on the lists of blocks. Several other implementations of this idea are possible, and some were discussed in Brown (1978). An array-based representation of the binomial heap structure was given in Strothotte and Sack (1985). One could also change the system of canonical sizes as long as we specify the block structure and the combination of a set of equal-sized blocks to a block of the next larger size; this gives a trade-off between the `insert` and the `delete_min` complexities that was studied in Fagerberg (1996b). Also the binomial heap structure formed the base of several other heaps, among them the Fibonacci heap (Fredman and Tarjan 1987) described in Section 5.8, the pairing heap (Fredman et al. 1986), and the relaxed heap (Driscoll et al. 1988). The pairing heap was especially popular for some time because it is easier to code; it is essentially related to the binomial heap in the same way as the skew heap is related to the leftist heap: a self-adjusting version in which

no balance conditions are checked and updated. It has amortized $O(\log n)$ bounds instead of the worst-case bounds of the binomial heap. The pairing heap was the object of several experimental studies as well as theoretical bounds (Stasko and Vitter 1987; Liao 1992; Fredman 1998, 1999a; Iacono 2000; Pettie 2005). A parametrized variant of these structures was discussed in Elmasry (2004). A general transformation of binomial-heap-like structures that defers comparisons and sometimes makes the structure more efficient by this was studied in Fredman (1999b).

All these structures used half-ordered trees, or equivalently heap-ordered trees of variable degree, as the underlying model. A structure that combines this idea of a list of canonical building blocks with (binary) heap-ordered trees instead is the M-heaps (Bansil, Sreekanth, and Gupta 2003), which use a list of complete binary heap-ordered trees as block structure, with the block heights in increasing order and all distinct except possibly the first two. Then in an `insert`, one joins together the two blocks of the same height, if they exist, or creates a new block of height 0, if not. This structure again allows an $O(\log n)$ worst-case `insert` and `delete_min`.

Any of these heaps based on binary addition of blocks again allows the amortized analysis that gives an $O(1)$ amortized complexity for the `insert` operation.

5.7 Changing Keys in Heaps

There is an additional operation on heaps that received much interest and was the main motivation for the interest in binomial heaps and their various relatives, which is to change the keys of elements, especially to decrease keys, which is necessary for Dijkstra's single-source shortest path algorithm and many combinatorial optimization algorithms.

This operation is different from the other operations we discussed so far because we have to identify the element that we want to change. A heap does not support a `find` operation, so we need a pointer into the structure to the element, a *finger* as in the finger search trees. This finger is returned by the `insert` operations and must refer to the element until it is deleted. In any actual implementation this requires some care because the node that contains the element possibly changes during the operations on the heap:

- { In the array-based heap, the item moves through the array.
- { If we use rotations as rebalancing method on half-ordered trees, our standard rotations copy the item to a different node.